



SEGUNDO PARCIAL – 08/11/2024

APELLIDO Y NOMBRE:LU:

CANTIDAD DE HOJAS ENTREGADAS (SIN ENUNCIADO):

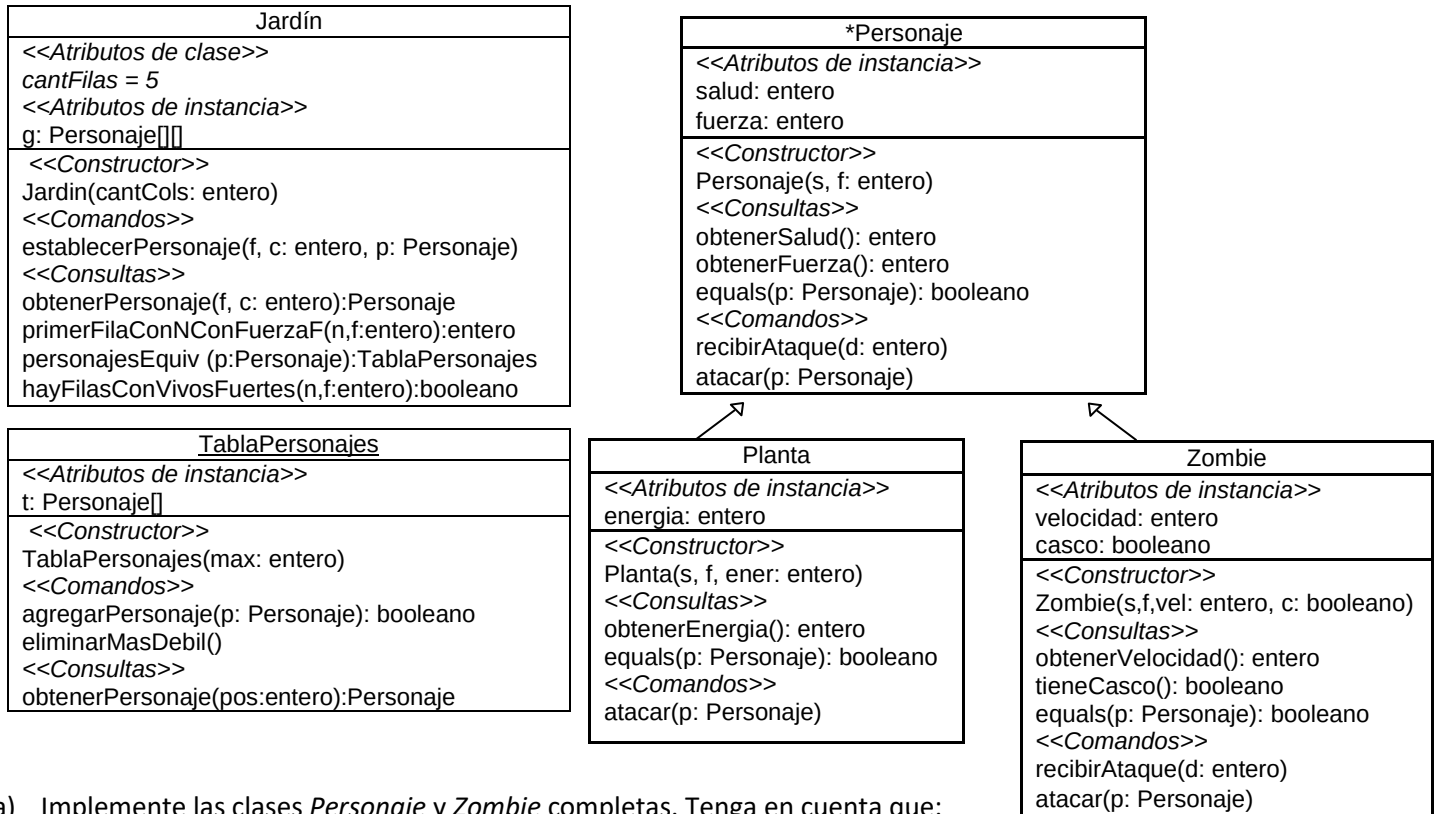
Importante: La interpretación del enunciado es parte del examen.

LA EVALUACIÓN SE REALIZARÁ CONSIDERANDO LA CORRECTITUD, LEGIBILIDAD Y EFICIENCIA DE LAS SOLUCIONES PROPUESTAS.

NO RESUELVA LOS EJERCICIOS EN EL ENUNCIADO. REALICE LOS PROBLEMAS EN HOJAS SEPARADAS.

PROBLEMA 1

Dado el siguiente diagrama de clases para una versión modificada del juego Plantas vs Zombies:



a) Implemente las clases *Personaje* y *Zombie* completas. Tenga en cuenta que:

En la clase **abstracta Personaje**:

- *Personaje(s, f: entero)*. Requiere $s > 0$ y $f \geq 0$.
- *equals(p: Personaje): booleano*. Dos personajes son equivalentes **si son del mismo tipo** y tienen los mismos valores en sus atributos de instancia. **Requiere p ligado**.
- *recibirAtaque(d: entero)*. Decrementa la salud del personaje receptor del mensaje en la cantidad indicada por el daño d . La salud nunca puede quedar negativa, en cuyo caso se establece en 0. Requiere $d > 0$.
- *atacar(p: Personaje)*. Requiere p ligado. Si la fuerza de p es menor que la fuerza del personaje que recibe el mensaje, el personaje p recibe un ataque con un daño **igual** a la fuerza del personaje receptor del mensaje. En caso contrario, no se produce ataque alguno.

En la clase **Zombie**:

- *Zombie(s, f, veloc: entero, c: booleano)*. Requiere $s > 0$, $f \geq 0$ y $veloc > 0$.
- *equals(p: Personaje): booleano*. Dos zombies son equivalentes si son del mismo tipo y tienen los mismos valores en sus atributos de instancia.
- *recibirAtaque(d: entero)*. Si el zombie receptor del mensaje no tiene casco, recibe el ataque como cualquier personaje. En caso contrario, decrementa su salud en la **mitad** de la cantidad indicada por d . La salud nunca puede quedar negativa, en cuyo caso se establece en 0. Requiere $d > 0$.

- *atacar(p: Personaje)*. Requiere *p* ligado. Si el objeto que recibe el mensaje tiene casco y *velocidad* > 2, el personaje *p* recibe un ataque con un daño igual al **doblo** de la fuerza del receptor del mensaje. En caso contrario, el objeto que recibe el mensaje ataca a *p* como cualquier personaje.

b) Implemente la clase **TablaPersonajes** completa que implementa una tabla de personajes. Tener en cuenta:

- *TablaPersonajes(max:entero)*. Requiere *max*>0.
- *agregarPersonaje(p:Personaje):booleano*. Requiere que la tabla no esté llena. Si *p* está ligado, asigna *p* a la primera posición libre y devuelve verdadero, caso contrario, devuelve falso.
- *eliminarMasDebil()*. Elimina de la tabla el personaje con menor fuerza.
- *obtenerPersonaje(pos:entero):Personaje*. Si *pos* es válida, retorna el personaje de la posición *pos*, sino retorna nulo.

c) Implemente la clase **Jardin** completa. Tenga en cuenta que:

- El constructor crea una matriz de *cantFilas* filas por *cantCols* columnas. Requiere *cantCols*>0.
- *establecerPersonaje(f,c: entero, p:Personaje)*. Asigna el personaje *p* (o nulo si *p* no está ligada) a la posición indicada por *f* y *c*. Requiere *f* y *c* válidos.
- *obtenerPersonaje(f,c: entero):Personaje*: Si *f* y *c* son válidos, devuelve el contenido de la posición indicada por *f* y *c*, sino retorna nulo.
- *primerFilaConNConFuerzaF(n,f:entero):entero*. Retorna el número de la primera fila con **al menos** *n* personajes con fuerza igual a *f*. Si no existe ninguna fila que cumpla con la condición retorna -1. Requiere *n,f*>0.
- *personajesEquiv (p:Personaje):TablaPersonajes*. Si *p* está ligado, retorna una tabla de tamaño igual a *cantCol* x *cantFilas* de *g*, con todos los personajes equivalentes a *p* ocupando las primeras posiciones de la tabla; caso contrario retorna nulo.
- *hayFilasConVivosFuertes(n,f:entero):booleano*. Devuelve *true* si y sólo si hay **exactamente** *n* filas con **al menos** un personaje con salud > 0 y fuerza > *f*. En caso contrario devuelve *false*. Requiere 0<*n*<=5.

PROBLEMA 2

Indique y justifique adecuadamente cuáles de las siguientes declaraciones de variables e instrucciones provocarán errores, indique el tipo de error (compilación o ejecución). Siempre que sea posible, muestre cuál sería la salida por consola.

<pre> abstract class Alfa{ protected int a; public Alfa(){ a = 1; } public String m(){ return "m en Alfa "+a+" "+n(); } public String n(){ return "n en Alfa"; } } </pre>	<pre> 1. Alfa a1, a2, a3; 2. Beta b1, b2; 3. Delta c1, c2, c3; 4. a1 = new Alfa(); 5. a2 = new Beta(); 6. b1 = new Beta(); 7. c1 = new Beta(); 8. b2 = new Delta(5); 9. c2 = (Delta) b2; 10. a3 = new Delta(15); 11. c3 = null; 12. System.out.print(a2.n()); 13. System.out.print(b2.n()); 14. System.out.print(a2.p()); 15. System.out.print(a2.m()); </pre>
<pre> class Beta extends Alfa{ protected int b; public Beta(){ super(); b = 10; } public String p(){ return "p en Beta "+ b; } public String n(int p){ return "n en Beta "+p*b; } } </pre>	<pre> 16. System.out.print(b1.n()); 17. System.out.print(c2.n(2)); 18. System.out.print(b1.p()); 19. System.out.print(c3.m()); 20. System.out.print(a3.m()); </pre>
<pre> class Delta extends Beta{ protected int c; public Delta (int n){ super(); c = n; } public String m(){ String s = super.m(); return "m en Delta "+c+" "+s; } public String n(){ return "n en Delta "+c*b; } } </pre>	